

Week-5 Assignment

Q1: What are the key limitations of traditional MapReduce that make **Spark** a preferred choice for modern big data processing?

Answer:

- MapReduce writes intermediate results to disk after every Map and Reduce phase, causing massive I/O overhead
- No support for iterative processing, making ML algorithms extremely slow
- Does not support real-time or interactive querying
- Complex pipelines require chaining multiple separate jobs, each reading and writing from HDFS, multiplying the latency

Q2: Explain how Spark uses **In-Memory Computing** to speed up iterative machine learning algorithms compared to disk-based systems.

Answer:

- In MapReduce, each iteration reads from disk, processes, and writes back to disk — 100 iterations mean 200 disk operations
- Spark loads data into memory once and keeps it there across all iterations
- Each iteration reads from RAM instead of disk, which is 10–100x faster
- This is why Spark MLlib trains models dramatically faster than MapReduce-based systems

Q3: Write a code snippet to remove all **duplicate rows** from a DataFrame based on a specific set of columns: `user_id` and `transaction_date`.

Answer:

```
df = df.dropDuplicates(["user_id", "transaction_date"])
```

Q4: Given a DataFrame `df_sales`, write a query to **filter** for rows where the region is 'West' and then **group by** `product_category` to find the average `sale_amount`.

Answer:

```
df_sales.filter(col("region") == "West") \
```

```
.groupBy("product_category") \
.agg(avg("sale_amount").alias("avg_sale_amount")) \
.show()
```

Q5: What is the difference between `.na.drop()` and `.na.fill()`? Provide a code example of filling null values in a status column with the string 'Unknown'.

Answer:

- `.na.drop()` removes the entire row that contains null values
 - `.na.fill()` replaces null values with a specified default, keeping the row intact
 - Use drop when nulls are unacceptable, use fill when a default value makes sense
- ```
df = df.na.fill({"status": "Unknown"})
```

**Q6:** Write a query to find the **total count** of records for each city in a DataFrame, but only for cities where the count is greater than 100.

Answer:

```
df.groupBy("city") \
.count() \
.filter(col("count") > 100) \
.show()
```

**Q7:** How does the **immutability** of Spark DataFrames affect how you perform "data cleaning" steps like dropping columns or renaming them?

Answer:

- Every cleaning operation like `drop()` or `withColumnRenamed()` does not modify the original DataFrame — it returns a new one
- You must always assign the result back to a variable, otherwise the change is lost
- Chaining multiple cleaning steps is safe because the original always remains intact in its lineage
- Spark tracks all transformations in a DAG, so any step can be recomputed if needed

**Q8:** Write a Spark command to **filter** a dataset for rows where the age is between 18 and 30 (inclusive) and the subscription is 'Premium'.

Answer:

```
df.filter(
 (col("age") >= 18) &
 (col("age") <= 30) &
 (col("subscription") == "Premium")
).show()
```

**Q9:** When **cleaning a dataset**, why is it often better to handle null values before performing mathematical aggregations like sum() or avg()?

Answer:

- sum() and avg() skip null values by default with no warning
- This can produce misleading results — e.g. an average over 800 rows when the DataFrame has 1000
- Handling nulls first makes the aggregation intentional and results accurate
- Either fill nulls with a default (e.g. 0) or drop them depending on the use case

**Q10:** Write the code to **revise** a column named raw\_timestamp by casting it to a TimestampType and renaming it to event\_time.

Answer:

```
from pyspark.sql.functions import col

from pyspark.sql.types import TimestampType

df = df.withColumn("raw_timestamp",
col("raw_timestamp").cast(TimestampType())) \

 .withColumnRenamed("raw_timestamp", "event_time")
```

**Q11:** Explain the "Shuffle" process that occurs during a **grouping** operation. Why is it considered a wide transformation?

Answer:

- When `groupBy()` is called, Spark needs all rows with the same key to land in the same partition
- Data is redistributed across the network to achieve this — this is called a shuffle
- It is a wide transformation because one output partition depends on data from multiple input partitions
- Shuffle involves cross-node communication, serialization, and temporary disk writes
- This is why `groupBy`, `join`, and `distinct` are expensive compared to narrow transformations like `filter`

**Q12:** Write a code snippet that identifies and **removes rows** where the email column contains null values OR the username is an empty string.

Answer:

```
from pyspark.sql.functions import col, trim

df = df.filter(

 col("email").isNotNull() &

 (trim(col("username")) != "")

)
```

**Q13:** How do you use the `.agg()` function to calculate multiple statistics at once, such as the min, max, and mean of the price column?

Answer:

```
from pyspark.sql.functions import min, max, avg

df.agg(

 min("price").alias("min_price"),

 max("price").alias("max_price"),

 avg("price").alias("mean_price")

).show()
```

**Q14:** In the context of **cleaning a dataset**, what is the risk of using `inferSchema=true` when your source data contains messy or inconsistent date formats?

Answer:

- Spark samples only a portion of data to guess types, so inconsistent formats can fool it
- A date column with mixed formats like "2024-01-15" and "15/01/2024" may be inferred as `StringType`
- Spark may silently produce nulls when it hits an unexpected format mid-dataset
- Errors only surface at execution time, not at load time, making them hard to trace
- Using `inferSchema=False` and casting manually gives full control over every column type

**Q15:** Write a final processing pipeline that:

1. Filters out **duplicates**.
2. Fills **null** prices with 0.
3. Groups by `store_id` to calculate total revenue.

Answer:

```
from pyspark.sql.functions import sum
result = (
 df.dropDuplicates()
 .fillna({"price": 0})
 .groupBy("store_id")
 .agg(
 sum("price").alias("total_revenue")
)
)
result.show()
```